



# **ANALYSIS AND DELETION OF YOUTUBE ABUSIVE COMMENTS**



**A MINI PROJECT II REPORT**

*Submitted by*

**DEEPSHIKA M (71812253011)**

**SUHITA M (71812253052)**

*in partial fulfillment for the award of the degree*

*of*

**MASTER OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**

**SRI RAMAKRISHNA ENGINEERING COLLEGE, COIMBATORE**

**ANNA UNIVERSITY: CHENNAI 600025**

**May 2025**

# **Computer Science and Engineering**

## **VISION**

To provide quality technical education and develop professionals imparting human values, employability, entrepreneurship and research capabilities, to meet the challenges in the globalized technological society.

## **MISSION**

- To enrich the students knowledge across the subject areas of computer science and engineering.
- To prepare students for careers in industry, encourage entrepreneurship and mould them to take leadership for the betterment of the society.
- To impart effective capabilities for the development of quality technical manpower to meet the real world challenges.

## **Program Educational Objectives (PEOs)**

**PEO I:** Work productively as computer engineers and succeed in diverse career path to solve real world problems.

**PEO II:** Collaborate efficiently with colleagues and be leaders in their profession with social and ethical responsibilities.

**PEO III:** Engage themselves in life-long learning to adapt with the continuously evolving technology.



**SRI RAMAKRISHNA ENGINEERING COLLEGE  
COIMBATORE**



**ANNA UNIVERSITY: CHENNAI 600025**

**BONAFIDE CERTIFICATE**

Certified that this project report entitled "Analysis and deletion of YouTube Abusive Comments" is the bonafide work of DEEPSHIKA M (71812253011), SUHITA M (71812253052), who carried out the 20CS296 MINI PROJECT II under my supervision.

*R.S.P. 5/5/25*  
**SIGNATURE**

**SUPERVISOR**

**Mrs. R. S. Ramya,**  
Assistant Professor (Sl. Grade),  
Department of Computer Science  
and Engineering,  
Sri Ramakrishna Engineering College,  
Coimbatore-641022.

*Ms. Geetha 5/5/25*  
**SIGNATURE**

**HEAD OF THE DEPARTMENT**

**Dr. M. S. Geetha Devasena, Ph.D.,**  
Professor and Head,  
Department of Computer Science  
and Engineering,  
Sri Ramakrishna Engineering College,  
Coimbatore-641022.

Submitted for the Project Viva-Voce Examination held on 05.05.2025

*Sham 5/5/25*  
**Internal Examiner**

*Dr. T. Rajasen 5/5/25*  
**External Examiner**

*Dr. T. Rajasen*

## DECLARATION

We affirm that the Mini Project II titled “ANALYSIS AND DELETION OF YOUTUBE ABUSIVE COMMENTS” being submitted in partial fulfillment for the award of Master of Technology is the original work carried out by us. It has not formed the part of any other project work submitted for award of any degree or diploma, either in this or any other University.

Suhita M

M. Datta

(Signature of the Candidates)

DEEPSHIKA M

(71812253011)

SUHITA M

(71812253052)

I certify that the declaration made above by the candidates is true.

R. S. P. S. Ramya

(Signature of the guide)

Mrs. R. S. Ramya,  
Assistant Professor (Sl. Grade),  
Department of CSE

## ACKNOWLEDGEMENT

We have immense pleasure in expressing our wholehearted thankfulness to **Shri.R.Sundar**, Managing Trustee, SNR Sons Charitable Trust and **Shri. S. Narendran**, Joint Managing Trustee, SNR Sons Charitable Trust for giving us the opportunity to study in our esteemed college and for very generously providing more than enough infrastructural facilities for us to get molded as a complete engineer.

We wish to express our profound and sincere gratitude to **Dr. A. Soundarrajan**, Principal, for inspiring us with his Engineering Wisdom. We also wish to record our heartfelt thanks for motivating and guiding us to become Industry ready engineers with inter-disciplinary knowledge and skillset with his multifaceted personality.

We extend our indebted thankfulness to **Dr. M. S. Geetha Devasena**, Ph.D., Professor and Head, for guiding and helping us in all our activities to develop confidence and skills required to meet the challenges of the industry. We also express our gratitude for giving us support and guidance to complete the project duly.

We owe our deep gratitude to our project coordinator **Dr. S. Harihara Gopalan**, Ph.D., Associate Professor, Department of Computer Science and Engineering who took keen interest in our project work and guided us all along with all the necessary inputs required to complete the project work.

We express our sincere thanks to our project supervisor **Mrs. R. S. Ramya**, Assistant Professor (Sl.Grade), the evaluators teaching faculty members and supporting staff members of the department for evaluating the project and providing valuable suggestions for improvements.

## TABLE OF CONTENTS

| CHAPTER NO. | TITLE                                         | PAGE NO. |
|-------------|-----------------------------------------------|----------|
|             | ABSTRACT                                      | VII      |
|             | சுருக்கம்                                     | VIII     |
|             | LIST OF FIGURES                               | IX       |
|             | LIST OF TABLES                                | X        |
|             | LIST OF ABBREVIATIONS                         | XI       |
| 1           | INTRODUCTION                                  | 1        |
|             | 1.1 Natural Language Processing               | 2        |
|             | 1.2 Web Development (With Streamlit)          | 3        |
|             | 1.3 Google Cloud Console and YouTube Data API | 4        |
| 2           | LITERATURE REVIEW                             | 6        |
| 3           | SYSTEM ANALYSIS                               | 12       |
|             | 3.1 Existing System                           | 12       |
|             | 3.2 Proposed System                           | 12       |
| 4           | PROBLEM IDENTIFICATION                        | 13       |
|             | 4.1 Problem Identification                    | 13       |
|             | 4.2 Problem Statement                         | 13       |
|             | 4.3 Methodology                               | 14       |
|             | 4.3.1 Data Preparation and Preprocessing      | 14       |
|             | 4.3.2 Model Fine Tuning Using Albert          | 14       |

|          |                                                          |    |
|----------|----------------------------------------------------------|----|
| 4.3.3    | Streamlit Web Application Development                    | 14 |
| 4.3.4    | OAuth Authentication And Google Cloud Setup              | 15 |
| 4.3.5    | YouTube Comment Retrieval And Moderation                 | 15 |
| 4.4      | Plan Of Experiments                                      | 15 |
| 4.5      | Theoretical Background                                   | 15 |
| 4.6      | Concepts                                                 | 17 |
| <b>5</b> | <b>DESIGN IMPLEMENTATION</b>                             | 18 |
| 5.1      | Workflow                                                 | 18 |
| 5.2      | Methodology                                              | 18 |
| 5.2.1    | Model Architecture And Fine Tuning With Albert           | 19 |
| 5.2.2    | Web Application Design Using Streamlit                   | 21 |
| 5.2.3    | OAuth Authentication And YouTube Data Api<br>Integration | 23 |
| 5.2.4    | Comment Classification And Moderation                    | 25 |
| <b>6</b> | <b>RESULTS AND DISCUSSION</b>                            | 27 |
| 6.1      | Testing Methods                                          | 27 |
| 6.2      | Testing Standards                                        | 28 |
| 6.3      | Sample Preparation                                       | 29 |
| 6.4      | Discussion                                               | 29 |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>7</b> | <b>CONCLUSION SCOPE FOR FUTURE WORK</b> | <b>33</b> |
|          | 7.1 Conclusion                          | 33        |
|          | 7.2 Scope For Future Work               | 34        |
| <b>8</b> | <b>REFERENCES</b>                       | <b>35</b> |
|          | <b>ANNEXURE I</b>                       | <b>36</b> |
|          | <b>APPENDICES</b>                       | <b>36</b> |
|          | 8.1 Source Code                         | 36        |
|          | 8.1.1 Training Albert Model             | 36        |
|          | 8.1.2 Streamlit Web Application         | 38        |
|          | 8.2 Output Snapshots                    | 42        |
|          | <b>ANNEXURE II</b>                      | <b>45</b> |
|          | 8.3 Base Paper                          | 45        |
|          | <b>ANNEXURE III</b>                     | <b>46</b> |
|          | 8.4 Paper Publication                   | 46        |
|          | <b>ANNEXURE IV</b>                      | <b>47</b> |
|          | 8.5 Plagiarism Report                   | 47        |

## **ABSTRACT**

In recent years, online platforms such as YouTube have become primary channels for content sharing, communication, and public engagement. However, the open nature of these platforms has also led to an increase in abusive, offensive, and inappropriate comments, which can adversely affect both creators and viewers. This project presents a semi-automated system for detecting and moderating abusive comments on YouTube videos using Natural Language Processing (NLP) and machine learning techniques. The system utilizes a fine-tuned ALBERT (A Lite BERT) model trained on a balanced dataset of abusive and non-abusive comments to perform binary classification. The web-based interface, built using Streamlit, allows YouTube content creators to authenticate securely using OAuth 2.0 credentials, input video IDs, fetch comments via the YouTube Data API, and classify each comment as either abusive or safe. If deemed abusive, the comment can be deleted upon user confirmation. The system was tested through real-time moderation of YouTube comments and through a dataset of 1,000 labeled samples. Results showed high prediction accuracy and reliable integration of model, API, and interface components. This solution not only reduces the manual burden of moderation but also ensures that creators remain in full control of content deletion. The project highlights the potential of combining deep learning and intuitive user interfaces to assist creators in maintaining respectful and healthy online communities.

## சுருக்கம்

சமீப காலங்களில் YouTube போன்ற தளங்களில் தவறான மற்றும் புண்படுத்தும் கருத்துகள் அதிகரித்துள்ளன, இது பயனர்களையும் படைப்பாளர்களையும் பாதிக்கிறது. இந்தத் திட்டம், ALBERT மாடலை அடிப்படையாகக் கொண்டு இயற்கை மொழி செயலாக்கம் மற்றும் இயந்திரக் கற்றலைப் பயன்படுத்தி YouTube கருத்துக்களை தவறானவையாக வகைப்படுத்தும் அரை தானியங்கி அமைப்பை உருவாக்குகிறது. Streamlit மூலம் உருவாக்கப்பட்ட இணைய இடைமுகம் மூலம், பயனர்கள் OAuth 2.0 மூலம் உள்நுழைந்து, வீடியோ கருத்துக்களைப் பெற மற்றும் மதிப்பீடு செய்ய முடியும். தவறான கருத்துகள் பயனர் உறுதிப்படுத்தலின் பேரில் நீக்கப்படலாம். இந்த அமைப்பு உயர்ந்த துல்லியத்துடன் செயல்படுகிறது மற்றும் மறைமுகமான கட்டுப்பாட்டைக் கொடுப்பதன் மூலம் மரியாதைமிக்க சமூகத்தை உருவாக்க உதவுகிறது.

## LIST OF FIGURES

| FIGURE NO. | NAME OF THE FIGURE          | PAGE NO. |
|------------|-----------------------------|----------|
| 5.1        | Flow Diagram                | 18       |
| 6.1        | Confusion matrix            | 30       |
| 6.2        | Graph Analysis              | 31       |
| 8.1        | Source Code Execution       | 42       |
| 8.2        | Uploading OAuth Credentials | 42       |
| 8.3        | Authentication Process      | 43       |
| 8.4        | Comments before Analysis    | 43       |
| 8.5        | Moderation Results          | 44       |
| 8.6        | Comments After Moderation   | 44       |

## **LIST OF TABLES**

| <b>TABLE NO.</b> | <b>NAME OF TABLE</b> | <b>PAGE NO.</b> |
|------------------|----------------------|-----------------|
| 6.1              | Model Analysis       | 31              |

## **LIST OF ABBREVIATION**

| <b>ABBREVIATION</b> | <b>EXPANSION</b>                                                      |
|---------------------|-----------------------------------------------------------------------|
| NLP                 | Natural Language Processing                                           |
| ALBERT              | A Lite BERT (Bidirectional Encoder Representations from Transformers) |
| SVM                 | Support Vector Machine                                                |
| Bi-LSTM             | Bidirectional Long Short-Term Memory                                  |
| CSV                 | Comma Separated Values                                                |
| OAuth 2.0           | Open Authorization 2.0                                                |

# CHAPTER 1

## INTRODUCTION

In today's digital era, YouTube has evolved into one of the most powerful platforms for content creation, sharing, and community engagement. With over two billion logged-in monthly users, it provides a space for education, entertainment, personal expression, and global communication. However, this open nature of interaction often comes with drawbacks, particularly the prevalence of harmful and abusive comments. These toxic interactions can significantly affect the mental well-being of content creators and disrupt meaningful discourse among viewers, leading to a less welcoming online environment.

While YouTube provides basic moderation tools and comment filters, they are often not sufficient to deal with the growing volume and complexity of abusive content. Manual moderation of thousands of comments is a tedious and time-consuming task, especially for popular creators and channels with wide reach. It's not only inefficient but also prone to inconsistency and human error. As a result, there is a growing demand for intelligent systems that can assist or even fully automate the moderation process with high accuracy and minimal human intervention.

To address this challenge, our project introduces an automated solution for detecting and moderating abusive comments on YouTube using advanced Natural Language Processing (NLP) and machine learning techniques. The core of our system is built using a fine-tuned ALBERT model (A Lite BERT), which is known for its efficiency and strong performance on language understanding tasks. This model is trained to classify comments as either "Safe" or "Abusive" based on patterns learned from a labeled dataset. Once trained, it can be deployed to evaluate real-world YouTube comments with high accuracy.

The project is implemented as a web-based application using Streamlit, a Python framework that simplifies the deployment of machine learning applications with interactive user interfaces. Users start by uploading a `client_secret.json` file from their Google Cloud Console, which is used to authenticate with YouTube via OAuth 2.0. They then input a YouTube video ID into the app. Upon running the application, it automatically fetches comments from the specified video, classifies each comment using the ALBERT model, and removes those identified as abusive by setting their moderation status to "rejected."

By combining machine learning with YouTube's API capabilities, this system aims to make comment moderation faster, smarter, and more reliable. It reduces the burden on creators while helping maintain a more respectful and positive environment in the comment sections. The application not only saves time but also enhances user experience by automating the moderation process in a scalable way. In doing so, it contributes to the broader goal of making online communities safer and more inclusive for all.

## **NATURAL LANGUAGE PROCESSING (NLP)**

Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on enabling machines to understand, interpret, and generate human language. It combines computational linguistics with statistical, machine learning, and deep learning models to process and analyze large amounts of natural language data. NLP allows computers to perform tasks such as language translation, sentiment analysis, speech recognition, and text classification. It has become essential in the development of intelligent systems that can interact with users in a more human-like manner.

A fundamental aspect of NLP is text preprocessing, which involves preparing raw text for analysis. This process includes steps like tokenization, stemming, lemmatization, stop-word removal, and normalization. These techniques help reduce the complexity of text data while retaining its essential features, allowing models to better understand the linguistic structure. Preprocessing plays a critical role in ensuring the quality and accuracy of NLP models, especially when dealing with diverse and unstructured text from real-world sources such as social media or user-generated content.

Modern NLP models are predominantly based on deep learning, with transformer architectures being the most influential in recent years. Transformers, such as BERT (Bidirectional Encoder Representations from Transformers), have revolutionized NLP by introducing attention mechanisms that capture the relationships between all words in a sentence simultaneously. This has led to a significant leap in performance for various tasks, as these models can understand context in a bidirectional way, unlike earlier approaches which processed text from one direction only.

ALBERT (A Lite BERT) is a more compact and efficient version of BERT that maintains the core advantages of the transformer architecture while reducing the computational overhead. It introduces parameter reduction strategies such as cross-layer parameter sharing

and factorized embeddings, making it faster and lighter for training and inference. Despite having fewer parameters, ALBERT retains high accuracy across a wide range of NLP benchmarks, making it an ideal choice for real-time and resource-constrained applications.

The use of NLP and models like ALBERT has expanded rapidly across industries. From customer service chatbots to healthcare record analysis, and from fraud detection to content moderation, NLP enables intelligent automation in numerous domains. Its ability to process and interpret large volumes of textual data not only boosts efficiency but also empowers systems to make more informed decisions based on nuanced human language. As language continues to evolve, NLP remains a vital tool for bridging the gap between human communication and machine intelligence.

## **WEB DEVELOPMENT (WITH STREAMLIT)**

Web development is the process of building and maintaining websites or web applications that are accessible via the internet. It involves both front-end development, which deals with user interfaces and user experience, and back-end development, which manages server-side logic and databases. The evolution of web technologies has made it easier to create interactive and dynamic applications that can run in browsers, offering real-time updates and data-driven content. Modern web development emphasizes responsive design, performance, and usability across various devices.

With the rise of data science and machine learning, there has been a growing need to create web applications that showcase analytical models and data visualizations. This has led to the development of Python-based web frameworks that simplify the process for developers who are more familiar with data analysis than traditional web programming. Streamlit is one such framework that allows data scientists to turn their Python scripts into interactive web apps with minimal effort, requiring no front-end experience in HTML, CSS, or JavaScript.

Streamlit is designed to be intuitive and efficient, offering built-in components like sliders, buttons, file uploaders, and charts. These widgets can be used to create engaging user interfaces for data-centric applications, all within a few lines of code. One of Streamlit's key features is its real-time reactivity—any change in input automatically triggers a rerun of the script, updating the output instantly. This makes it ideal for experimenting with machine learning models or presenting dynamic data dashboards.

Another advantage of Streamlit is its seamless integration with Python libraries commonly used in data science, such as NumPy, Pandas, Matplotlib, and TensorFlow. Developers can load models, process data, and generate predictions directly within the app. Furthermore, Streamlit supports cloud deployment through platforms like Streamlit Cloud, Heroku, or AWS, allowing apps to be shared publicly or privately with ease. This accessibility accelerates collaboration, demonstration, and prototyping in machine learning workflows.

Web development frameworks like Streamlit bridge the gap between data science and interactive application development. They empower researchers, engineers, and analysts to present their work in a more user-friendly and impactful way, without needing extensive front-end knowledge. As the demand for intelligent, data-driven applications continues to grow, tools like Streamlit will play an increasingly important role in bringing machine learning and data science to broader audiences via the web.

## **GOOGLE CLOUD CONSOLE AND YOUTUBE DATA API**

Google Cloud Console is a web-based platform that provides a user interface for managing resources and services within Google Cloud. It allows developers to configure APIs, manage authentication credentials, monitor usage, and control access to various cloud-based tools. With its wide range of services, including storage, machine learning, and networking, the Cloud Console is a central hub for deploying, scaling, and managing applications in the cloud. It supports both manual setup and automated scripting through its robust command-line interface and APIs.

One of the core components of working with Google APIs is authentication, most commonly achieved using OAuth 2.0. This protocol enables secure authorization without exposing user credentials. Developers can create credentials such as API keys or OAuth client IDs through the Cloud Console, which are then embedded in applications to access user-specific services. OAuth 2.0 is particularly valuable when working with APIs that require permission to interact with a user's private data, such as YouTube comments or Gmail messages.

The YouTube Data API is part of Google's suite of APIs that allow external applications to interact with YouTube services. It provides programmatic access to features like retrieving video metadata, posting comments, managing playlists, and moderating content. Developers

can use this API to automate repetitive tasks, gather insights, or build tools that enhance user experience. With its RESTful architecture and extensive documentation, the YouTube Data API is widely used in analytics, content management, and moderation tools.

To use the YouTube Data API, developers must enable it from the Google Cloud Console and generate credentials that grant their application permission to interact with the service. These credentials are securely stored in a `client_secret.json` file, which contains information like the client ID and secret used during the OAuth flow. Once authenticated, the API can be accessed through various client libraries or directly via HTTP requests. This setup ensures that all interactions with YouTube are secure, authorized, and logged.

Integrating Google Cloud services and APIs into applications opens up powerful possibilities for automation, scalability, and real-time data processing. Whether building content analysis tools, media management dashboards, or machine learning pipelines, the Google Cloud Console provides the necessary infrastructure to support these solutions. With robust authentication, detailed access controls, and seamless integration with other Google services, it forms the backbone of many cloud-native applications in both enterprise and personal projects.

## **CHAPTER 2**

### **LITERATURE REVIEW**

#### **2.1 “Detecting Abusive Comments at a Fine-Grained Level in a Low-Resource Language”, B.R. Chakravarthi, R. Priyadharshini, S. Banerjee et al., 2023**

Detecting abusive language in low-resource languages such as Tamil and Tamil-English code-mixed text presents unique challenges due to limited annotated data and linguistic complexity. To tackle this, the authors compile four diverse datasets of YouTube comments annotated for different categories of abuse, including misogyny, homophobia, and xenophobia. A fine-grained classification strategy is adopted to better capture the nuances of online toxicity. The study evaluates both traditional machine learning methods like Support Vector Machines (SVM) and Logistic Regression, and deep learning models such as Bi-LSTM and BERT-based transformers.

Results reveal that traditional machine learning models, especially SVM paired with TF-IDF features, perform better than deep learning models in this context, underscoring the influence of dataset quality and size over model sophistication. The study also compares feature extraction techniques like Bag of Words, TF-IDF, and FastText, with TF-IDF yielding the best classification accuracy. Ethical issues, including user privacy and the difficulty of labeling sensitive content, are thoughtfully addressed to ensure responsible dataset usage. By contributing dedicated datasets and evaluation benchmarks, the authors support further research in abusive comment detection for low-resource languages.

#### **Advantages**

- Provides valuable datasets for abusive comment detection in low-resource languages, aiding future research.
- Emphasizes ethical considerations in dataset creation, ensuring privacy and realistic language use.

- Demonstrates that traditional machine learning models can outperform deep learning models in low-data environments, offering practical solutions for real-world applications.

## **Disadvantages**

- Limited to low-resource languages, which might limit generalization to more widely spoken languages.
- The reliance on smaller datasets might affect the robustness of models in diverse real-world contexts.
- Fine-grained classification remains a challenge, as the accuracy of distinguishing specific forms of abuse depends on the quality of annotations and training data.

## **2.2 “A Research on YouTube Spam Comments Detection and Deletion”, Darshan Bhavsar, Stephen Dcruz, Abhishek Chandekar, Mihir Chaudhari, Prof. Chetana Patil, May 2023**

Spam comments on YouTube pose significant challenges, ranging from user disruption and reduced content visibility to potential security risks. These comments often involve self-promotion, phishing, malware links, and abusive content. To address this, the authors introduce an automated spam detection and deletion system using Natural Language Processing (NLP) techniques combined with string similarity metrics like Levenshtein Distance and Rapid Fuzz. These methods enhance the system's ability to detect modified or misspelled spam content that might otherwise bypass basic filters. The solution integrates with the YouTube Data API v3, enabling automatic retrieval and removal of suspicious comments.

The system uses various pre-processing strategies, including character normalization, removal of special symbols, and suspicious username detection, to improve accuracy. Scalability is achieved through multi-threading, allowing the tool to handle large volumes of comments efficiently. Empirical evaluations show that the system outperforms traditional models such as SVM and KNN in terms of precision and recall. However, despite its high performance, the system's reliance on static spam patterns limits its adaptability to emerging and evolving spam tactics, indicating the need for future improvements with more dynamic detection mechanisms.

## Advantages

- Combines classic string matching techniques with NLP for improved spam detection.
- Scalable approach with multi-threaded scanning, enabling faster and more efficient comment moderation.
- Secure and automated system that leverages the YouTube API for seamless integration into existing platforms.

## Disadvantages

- The approach is limited to detecting spam based on predefined templates, which may not account for new or evolving spam strategies.
- The reliance on string-based similarity may result in false positives or missed detections in complex comment structures.
- The method may be less effective in languages with significant slang or code-switching, which may not be well-represented in predefined templates.

### **2.3 “Abusive Language Detection in YouTube Comments Leveraging Replies as Conversational Context”, Noman Ashraf, Arkaitz Zubiaga, Alexander Gelbukh, October 2021**

To improve abusive language detection in YouTube comments, this research introduces a context-aware approach that considers both comments and their replies. Traditional models often fail to detect sarcasm, indirect insults, or tone-dependent abuse when analyzing comments in isolation. To address this, the authors present the CAALDYC dataset, which includes annotated comment-reply pairs, allowing models to interpret language within conversational context. This design enables more accurate identification of subtle or indirect abusive behavior that might otherwise appear neutral when viewed out of context.

The study evaluates multiple machine learning models, including Adaboost, Random Forest, and LSTM with attention mechanisms. The attention component allows the model to focus on key parts of the conversation, enhancing its ability to detect nuanced abuse. Results show improved performance over models that treat comments independently. However, the method increases computational complexity and may struggle with short or reply-less threads.

Additionally, sophisticated forms of abuse like psychological manipulation remain challenging to identify. Despite these limitations, the study marks a significant step toward developing more intelligent and context-sensitive moderation tools.

### **Advantages**

- Incorporating replies as context leads to improved accuracy in detecting abusive language.
- Offers a dataset specifically designed for abusive comment detection in conversational settings, enhancing model performance.
- Provides benchmarks for further research in abusive language detection in threaded discussions like YouTube comments.

### **Disadvantages**

- Context-based models may require more computational resources due to the need to process entire conversations.
- The accuracy may still be influenced by the quality and completeness of the dataset, especially for implicit forms of abuse.
- Analyzing replies may not always provide sufficient context for determining abusive intent, as tone and intent can be difficult to infer from text alone.

## **2.4 “YouTube Comments Sentiment Analysis”, Ritika Singh, Ayushka Tiwari, May 2021**

This study explores sentiment analysis of YouTube comments to understand public opinion and emotional trends, offering actionable insights for content creators, businesses, and marketers. The authors aim to classify comments into positive, negative, and neutral categories, tackling the informal and unstructured nature of online text through preprocessing techniques such as stop-word removal, tokenization, and lemmatization. A dataset of 1500 cleaned and annotated comments is used to evaluate the effectiveness of various machine learning models.

Multiple classifiers are tested, including Naïve Bayes, SVM, Logistic Regression, KNN, Decision Trees, and Random Forest. Results indicate that Decision Trees perform best with n-gram features for macro F-scores, while Random Forest achieves the highest micro-average F-scores. SVM maintains consistent performance across all metrics. N-gram features prove effective in capturing contextual meaning, enhancing classification accuracy. Visual

tools like word clouds highlight common sentiment expressions, reinforcing the model's interpretability. The study demonstrates that traditional machine learning techniques, when paired with strong preprocessing and feature engineering, are effective for large-scale sentiment analysis. Future research could benefit from integrating deep learning models and more advanced linguistic features.

### **Advantages**

- Provides an effective method for sentiment classification using traditional machine learning algorithms.
- Utilizes visual analytics (word clouds) to provide an intuitive understanding of comment sentiment.
- Demonstrates the effectiveness of n-grams and other feature engineering techniques in improving sentiment analysis performance.

### **Disadvantages**

- The reliance on traditional machine learning algorithms may limit the model's ability to handle complex linguistic patterns.
- The dataset size may not be sufficient to fully capture the variety of sentiment expressions in YouTube comments.
- The study focuses primarily on English-language comments, limiting its applicability to other languages or multilingual platforms.

## **2.5 “Analysing Sentiments for YouTube Comments using Machine Learning”, Sainath Pichad, Sunit Kamble, Rohan Kalamb, Sumit Chavan, May 2023**

This paper explores the use of machine learning and NLP techniques for sentiment analysis on YouTube comments to understand user sentiment and improve engagement strategies. The study aims to classify comments into binary (positive/negative) and multi-class (positive/neutral/negative) sentiments, while also capturing emotions like happiness, sadness, and anger. The challenges of informal language, slang, emojis, and abbreviations in YouTube comments require advanced NLP methods for accurate sentiment interpretation.

The authors employ a variety of techniques, including lexicon-based methods, machine learning models (Naïve Bayes, SVM, KNN), and deep learning models (RNN, LSTM). They propose a hybrid approach combining VADER (a lexicon tool) with LSTM networks to improve accuracy and reduce false predictions. Genetic algorithms are used to optimize hyperparameters and features.

The results show that the hybrid approach outperforms individual models, achieving accuracy up to 95.5%, with high precision, recall, and F1-scores. Visual tools like word clouds help identify dominant sentiment terms. The study highlights the power of machine learning for sentiment analysis on platforms like YouTube and suggests future improvements, including multi-language support, better sarcasm detection, and real-time analysis scalability.

### **Advantages**

- A hybrid methodology that combines lexicon-based tools with deep learning models improves sentiment classification accuracy.
- Genetic algorithms help optimize hyperparameters and feature selection, enhancing model performance.
- High accuracy and robust evaluation metrics make the approach suitable for real-world applications in sentiment analysis.

### **Disadvantages**

- The use of genetic algorithms for optimization may introduce additional computational complexity and time requirements.
- The model's performance may be limited by the quality and diversity of the training data.
- The focus on sentiment analysis alone may overlook other important aspects of comment analysis, such as toxicity or relevance.

## **CHAPTER 3**

### **SYSTEM ANALYSIS**

#### **3.1 EXISTING SYSTEM**

In the current digital landscape, especially on platforms like YouTube, managing user comments is a significant challenge. Existing systems rely on manual moderation, keyword filtering, and community reporting, often using rule-based mechanisms or pattern matching to detect abusive language. However, these systems are not always transparent, and their accuracy is inconsistent, especially when dealing with context-dependent or sarcastic language.

Keyword filters can be bypassed with minor alterations, slang, or hidden harmful intent within neutral language. Manual moderation is labor-intensive and impractical at scale, particularly when videos receive thousands of comments quickly.

Furthermore, developers and researchers face challenges in experimenting with or improving moderation systems due to limited access. Current systems don't easily allow for custom model training, dataset evaluation, or comment-level interaction via public APIs, hindering the development of real-time, tailored moderation tools for educational or research purposes.

#### **3.2 PROPOSED SYSTEM**

The proposed system provides a solution for detecting and managing abusive comments on YouTube via a Streamlit web application. It uses a fine-tuned ALBERT model to classify comments as abusive or non-abusive. Users authenticate securely with the YouTube API by uploading a `client_secret.json` file and inputting a YouTube video ID to retrieve comments.

The system processes the comments in real-time, classifying them based on the binary classification task. Abusive comments are flagged and can be deleted using the YouTube Data API, with deletion happening only after the user's confirmation, ensuring control and transparency.

The interactive web interface allows users to upload the authentication file, enter the video ID, and trigger comment analysis. The ALBERT model, trained on a labeled dataset, effectively identifies abusive content. The system offers an efficient, user-friendly solution for real-time comment moderation on YouTube videos.

## **CHAPTER 4**

### **PROBLEM IDENTIFICATION**

#### **4.1 PROBLEM IDENTIFICATION**

- YouTube creators often receive a large number of comments, making manual review and deletion of abusive content time-consuming and impractical. This delay can allow harmful comments to remain public for extended periods.
- YouTube's native comment filtering system is limited and often ineffective in identifying harmful or toxic language, especially when the abuse doesn't involve explicit keywords.
- There are currently no user-friendly tools that allow creators to automatically fetch, classify, and delete abusive comments on their videos using machine learning. Most moderation still requires manual effort or third-party services.
- Although APIs and models exist for abuse detection, there is no accessible web-based solution that integrates YouTube comment fetching, AI-based classification, and automatic deletion in a single streamlined interface.
- Managing API access and OAuth 2.0 authentication for YouTube data can be challenging for non-technical users. A guided web interface is essential to simplify the process and make the tool usable by a wider audience.

#### **4.2 PROBLEM STATEMENT**

- YouTube content creators are increasingly exposed to abusive and harmful comments, which negatively impact their mental well-being and audience engagement. Manual moderation is time-consuming and inefficient, especially for creators managing high-traffic videos.
- There is a lack of an accessible, web-based solution that empowers creators to automatically detect and delete abusive comments using machine learning, while also simplifying the technical processes like authentication and comment retrieval.

## **4.3 METHODOLOGY**

The methodology adopted in this project involves a structured approach to developing a machine learning-based web application for detecting and managing abusive comments on YouTube. The process is divided into multiple stages, each focusing on a specific functionality, from model development to API integration and user interface design.

### **4.3.1 Data Preparation And Preprocessing**

The first step in the methodology involves preparing a labeled dataset suitable for binary text classification where each comment is categorized as either abusive or non-abusive. This stage begins with collecting a large volume of real or synthetic YouTube-style comments and labeling them accordingly. The raw text data undergoes preprocessing techniques, including lowercasing, punctuation removal, stop-word removal, and tokenization. These steps are essential to clean and standardize the text for efficient model training. In addition, lemmatization is applied to convert words to their base forms, helping the model better understand language semantics. The dataset is then split into training and validation subsets to facilitate supervised learning and model evaluation.

### **4.3.2 Model Fine-Tuning Using Albert**

Once the dataset is ready, the next step is to fine-tune a pre-trained language model for the classification task. The model selected is ALBERT (A Lite BERT), a lightweight version of the BERT transformer model, optimized for speed and memory efficiency. The fine-tuning process involves feeding the preprocessed comment text into the ALBERT tokenizer, transforming the text into numerical tokens compatible with the model. A classification layer is added on top of the model to allow binary output for abuse detection. The training process uses a suitable loss function and optimizer to adjust the model weights according to the labeled examples. Once the model is fine-tuned, it is saved along with its tokenizer for use in the deployment phase.

### **4.3.3 Streamlit Web Application Development**

To make the system accessible and interactive, a front-end web application is developed using Streamlit, a Python framework designed for building data apps with minimal code. The Streamlit interface includes key components such as a file uploader for the `client_secret.json`

file, a text input for entering the YouTube video ID, and an action button to trigger comment analysis and moderation. The layout is organized to guide users through each step clearly. Behind the interface, Streamlit handles the logic for passing comments through the trained model and displaying the results. The framework supports real-time updates, allowing users to instantly view classification results and moderation status.

#### **4.3.4 OAuth Authentication And Google Cloud Setup**

A crucial aspect of the methodology involves setting up OAuth 2.0 authentication to securely access YouTube data on behalf of the user. The user must first generate a `client_secret.json` file from the Google Cloud Console. This involves creating a new project, enabling the YouTube Data API v3, and creating OAuth client credentials under the “Desktop App” category. In the web app, the user uploads this file, which is then used to initiate the OAuth flow using the `google-auth-oauthlib` library. Upon successful login and authorization, a secure connection is established with YouTube, allowing the application to perform actions such as retrieving video comments and modifying their moderation status.

#### **4.3.5 Youtube Comment Retrieval And Moderation**

After authentication, the application uses the YouTube Data API to fetch comments from the video associated with the input video ID. These comments are collected in real-time and extracted along with their unique comment IDs. Each comment is then passed through the previously trained ALBERT model for classification. Based on the prediction, the app labels comments as either abusive or safe. Importantly, the app does not automatically delete comments. Instead, once the user clicks the moderation button, the app proceeds to delete only those comments identified as abusive by calling the API method `setModerationStatus` with the status set to "rejected." All actions are executed only after the user's explicit confirmation to maintain control and transparency.

### **4.4 PLAN OF EXPERIMENTS**

The experimental phase of this project is planned in sequential steps to evaluate the effectiveness of a fine-tuned ALBERT model in classifying YouTube comments as abusive or non-abusive. The first stage involves preparing a balanced dataset comprising 3,000 abusive and 3,000 non-abusive labeled sentences. This dataset is divided into training and validation sets with an 80:20 ratio.

The model is trained using supervised learning in a controlled environment using TensorFlow. During training, hyperparameters such as learning rate, batch size, number of epochs, and dropout rate are selected empirically. Performance metrics including accuracy, precision, recall, and F1-score are tracked per epoch.

Post-training, the model is integrated into a Streamlit web application for real-world testing. Users authenticate via the YouTube Data API, fetch video comments, and trigger classification and deletion. Experimental runs are conducted by uploading different `client_secret.json` files and inputting various YouTube video IDs to test the end-to-end workflow. Each experimental cycle measures the number of comments fetched, the number classified as abusive, and the success rate of deletions.

## 4.5 THEORETICAL BACKGROUND

The theoretical foundation of this project lies primarily in Natural Language Processing (NLP), transfer learning, and transformer-based architectures. NLP enables machines to understand, interpret, and respond to human language in a meaningful way. In particular, the task of abusive comment detection falls under the domain of text classification, where a machine learning model learns to assign predefined categories to text data.

Transfer learning is used in the form of pre-trained language models. Instead of training a deep model from scratch, we fine-tune ALBERT—a variant of BERT (Bidirectional Encoder Representations from Transformers)—on a smaller, domain-specific dataset. ALBERT improves upon BERT by reducing the number of parameters through techniques like factorized embedding parameterization and cross-layer parameter sharing.

The Transformer architecture, which underpins ALBERT, uses a mechanism called self-attention to understand the relationships between words in a sentence, regardless of their position. This allows the model to understand context more effectively than traditional models. The classification head on top of ALBERT converts the contextual embeddings into binary predictions—either “abusive” or “safe.”

## 4.6 CONCEPTS

This project is built upon several core concepts from computer science and machine learning:

1. **Natural Language Processing (NLP):** The application of computational techniques to process and analyze human language. In this context, NLP is used for comment preprocessing, tokenization, and classification.
2. **Transformers and ALBERT:** ALBERT is a light-weight version of BERT that reduces the number of parameters, making it faster and more efficient. It retains the core idea of transformers, especially the self-attention mechanism that allows contextual understanding of words in a sentence.
3. **OAuth 2.0 Authentication:** A secure protocol used for authorizing third-party applications to access user data without sharing login credentials. This is used to access YouTube APIs in a secure and user-consented manner.
4. **YouTube Data API:** A RESTful API provided by Google that enables applications to retrieve YouTube content like video metadata and comments, and perform moderation tasks such as setting comment status.
5. **Web Application with Streamlit:** Streamlit is a Python framework for building interactive web apps. It enables rapid development of front-end interfaces for machine learning models, allowing real-time interaction with comment data and classification results.

## CHAPTER 5

### DESIGN IMPLEMENTATION

#### 5.1 WORKFLOW

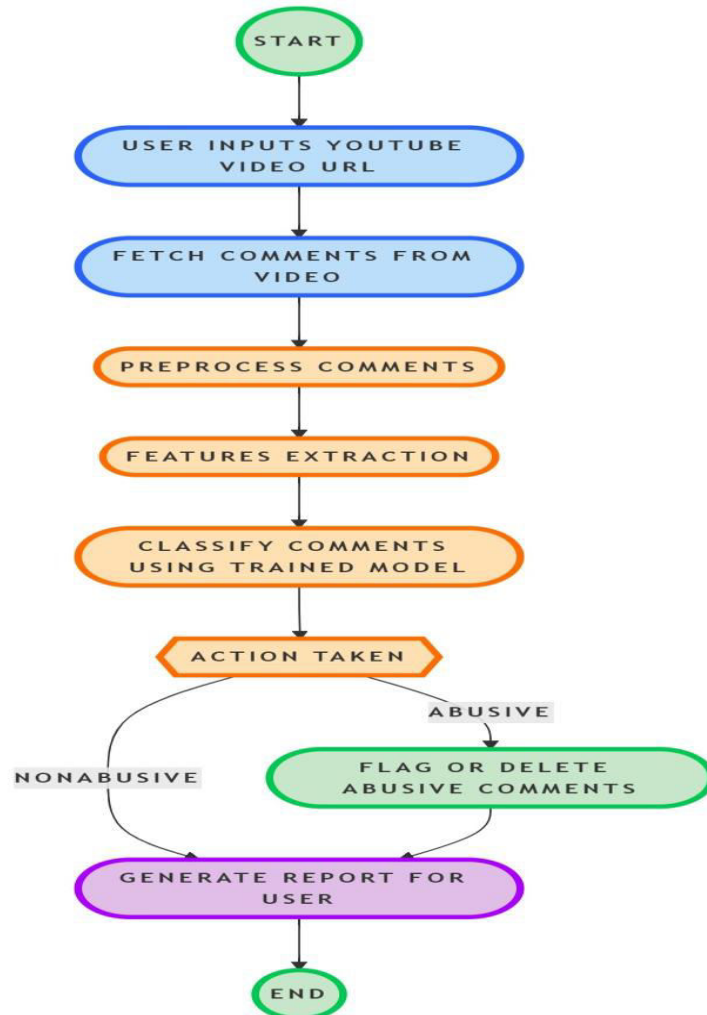


Fig.5.1 Flow Diagram

#### 5.2 METHODOLOGY

The implementation of the proposed system is carried out in several modular stages to ensure clarity, scalability, and maintainability. Each module focuses on a core aspect of the system, from model training to frontend integration.

### 5.2.1 Model Architecture And Fine-Tuning With Albert

The machine learning engine powering the abusive comment detection functionality is based on the **ALBERT (A Lite BERT)** model, a high-performance language representation model developed by Google. ALBERT is a member of the transformer family of models and was specifically designed to achieve the same levels of contextual understanding as BERT while significantly reducing computational overhead and memory usage. This is accomplished through two key innovations: **factorized embedding parameterization** and **cross-layer parameter sharing**, which reduce the number of model parameters while maintaining high accuracy in language understanding tasks.

ALBERT is particularly well-suited to applications that require both **deep contextual language modeling** and **real-time prediction**, such as the detection of abusive or toxic language in short-form user-generated text. In this system, ALBERT serves as the backbone for classifying YouTube comments. It has been fine-tuned using a **binary classification task**, where the goal is to determine whether a comment is *abusive* (1) or *safe* (0). This classification is made by analyzing the linguistic structure, tone, and word usage patterns in each comment.

The dataset used for fine-tuning consists of **6,000 labeled YouTube-style comments**, evenly distributed between the two classes. These comments mimic real-world comment behavior, containing informal language, slang, abbreviations, emojis, and variable sentence structures. Preprocessing begins with cleaning the raw text by removing special characters, converting to lowercase, and standardizing spaces. The data is then tokenized using the **AlbertTokenizer**, which transforms text into a format the model can understand by:

- Adding special tokens like [CLS] at the beginning and [SEP] at the end.
- Splitting words into subword units (wordpiece tokenization).
- Converting tokens into input IDs and generating attention masks.
- Padding or truncating sequences to a maximum length of 512 tokens.

After tokenization, the text is passed into the **pre-trained ALBERT base model**, which generates deep contextual embeddings. ALBERT processes each token through multiple attention layers, using self-attention mechanisms to compute the importance of each word in relation to others in the sentence. The final output of the model includes:

- A **pooled output** corresponding to the [CLS] token, which serves as a compact representation of the entire sentence and is used for classification.
- **Hidden states** for each token, which can be used for more advanced tasks like token-level predictions (not required in this use case).

For classification, a **dense (fully connected) layer** is appended to the pooled output. This layer applies a **softmax activation function** to output probabilities for the two classes (abusive or safe). The predicted class is determined by selecting the label with the highest probability. During training, the model is optimized using a **categorical cross-entropy loss function**, which compares the predicted class distribution with the actual label.

The model is fine-tuned using the **Hugging Face Transformers library** and **TensorFlow** backend. The following hyperparameters and configurations are used during training:

- **Batch Size:** 16
- **Epochs:** 10
- **Learning Rate:** 2e-5
- **Dropout Rate:** 0.3 (applied before the classification layer to reduce overfitting)
- **Optimizer:** Adam (with default  $\beta$  values for stability and convergence)

The training data is divided into training and validation sets using an 80:20 ratio to ensure the model generalizes well. During each epoch, the model's performance is monitored using accuracy, precision, recall, and F1-score. These metrics are tracked after every epoch to assess learning progress, identify underfitting or overfitting, and fine-tune the training process.

Once training is complete, both the model and tokenizer are saved using Hugging Face's `save_pretrained()` function. These artifacts are loaded directly into the Streamlit application during runtime using the corresponding `from_pretrained()` method, ensuring that the model used in deployment is identical to the one used during training. Input comments are dynamically passed through the tokenizer and model, and the output labels are interpreted to determine whether the text should be flagged for moderation.

The ALBERT model operates efficiently in real-time within the web app, requiring only a few milliseconds to process each comment. Its ability to interpret complex sentence structures

and its lightweight nature make it an optimal choice for abusive language detection on user-generated platforms like YouTube.

### 5.2.2 Web Application Design Using Streamlit

The front-end interface of the system was developed using **Streamlit**, a modern, open-source Python framework tailored for building interactive web applications focused on data science, machine learning, and real-time analytics. Streamlit offers a highly intuitive syntax and eliminates the need for traditional front-end technologies such as HTML, CSS, and JavaScript. This makes it an ideal choice for machine learning developers and researchers who want to create user interfaces without switching tech stacks. In this project, Streamlit plays a central role in facilitating user interaction with the fine-tuned ALBERT model and the YouTube Data API in a clean, responsive, and easy-to-use manner.

The interface was designed with usability and accessibility in mind, particularly for YouTube content creators who may not have any technical background. It provides a step-by-step guided workflow, making it easy for the user to authenticate with Google, fetch comments, run predictions, and trigger deletions. The layout is linear, with visual cues and instructional text guiding the user at every step. Every component was selected and implemented to fulfill a specific function in the user journey, ensuring seamless interaction with the underlying logic.

One of the first components the user encounters is the File Uploader Widget. This widget prompts users to upload their `client_secret.json` file, which is required for secure authentication using Google OAuth 2.0. This JSON file contains sensitive credentials, including the client ID and client secret issued by the Google Cloud Console. By uploading this file, the user enables the application to request permission to access their YouTube account data and perform comment moderation actions. Streamlit handles the file upload securely by temporarily saving the file to disk using Python's `tempfile` module, after which it is used during the authentication process.

Next, the user is presented with a Text Input Widget. This is a simple field where the user enters the YouTube Video ID for the video whose comments they want to scan. The application uses this ID to query the YouTube Data API and retrieve the associated top-level comments. Streamlit validates the field to ensure that the input is not left empty and displays a warning message if the user tries to proceed without entering a valid ID.

The application interface features two key Button Widgets. The first button triggers the end-to-end process of authenticating the user, retrieving comments via the YouTube Data API, passing those comments through the ALBERT model for classification, and displaying the results in the interface. The second button is used to initiate the deletion of abusive comments, but only after the classification step is complete and the user has explicitly chosen to proceed. This separation of prediction and deletion ensures that the user maintains control over what gets moderated on their video, avoiding unintended deletions.

The core output of the interface is displayed in the Results Panel, which dynamically renders each fetched comment alongside its predicted label. Comments classified as “Safe” are displayed in a neutral or positive color (e.g., blue or green), while those labeled as “Abusive” are highlighted in red with an accompanying warning icon or emoji. Each abusive comment is also accompanied by a status message indicating whether it was successfully deleted, skipped, or if an error occurred (e.g., due to lack of permissions or invalid comment ID). This panel provides transparency and feedback for every moderation action, enhancing the reliability of the system from the user's perspective.

Streamlit’s reactive programming model ensures that the entire interface responds automatically to user inputs. Under the hood, Streamlit reruns the script every time a widget's value changes. For example, uploading the credential file or entering a new video ID automatically re-triggers the logic associated with those inputs. This behavior allows for a smooth and interactive experience but also requires efficient handling of resources. To prevent redundant reloading of large resources such as the ALBERT model or the tokenizer, the application uses the decorators `@st.cache_resource` and `@st.cache_data`. These decorators instruct Streamlit to cache the output of functions like `load_model_and_tokenizer()` so that they are only re-executed when necessary.

Additionally, all backend logic—including model inference, API requests, comment parsing, and deletion—is modularized into clean, well-documented functions. This not only improves readability and maintainability of the codebase but also separates concerns, allowing the frontend to focus solely on user experience while the backend handles processing and integration tasks. Error handling mechanisms are also built into each function to catch and display API errors, invalid inputs, or failed deletions, ensuring robustness and reliability.

The Streamlit web app is designed to run locally or be deployed to the cloud using platforms like Streamlit Cloud, Heroku, or AWS EC2. When deployed, the app maintains the

same functionality, allowing remote users to access its features via a web browser. Overall, the use of Streamlit transforms a complex backend process involving machine learning and API calls into an approachable and efficient interface for real-world YouTube content moderation.

### **5.2.3 OAuth Authentication And Youtube Data Api Integration**

The core functionality of this project—fetching comments from YouTube videos and performing moderation actions such as deletion—requires secure access to a user’s YouTube account. This access is not granted through a simple API key due to the sensitive nature of these operations, such as managing user-generated content. Instead, Google mandates the use of OAuth 2.0, a widely accepted authorization protocol that allows applications to securely access user resources without handling their actual login credentials.

OAuth 2.0 ensures that the application acts on behalf of the user only with their explicit consent. In the context of this system, it allows the Streamlit application to read comments from a specific video and moderate (reject/delete) comments as long as the authenticated user has the appropriate permissions (e.g., they are the video owner). The authentication process begins with the generation of a `client_secret.json` file, which acts as a secure credential for initiating the OAuth 2.0 flow.

To generate this file, the user must first access the Google Cloud Console at <https://console.cloud.google.com>. From there, they need to perform the following steps:

1. Create a new project (or select an existing one) under their Google account to isolate API configurations.
2. Navigate to “Library” and enable the YouTube Data API v3, which provides access to YouTube resources like comments, videos, and channels.
3. Go to the “Credentials” tab and select “Create Credentials” > “OAuth Client ID”.
4. Choose “Desktop App” as the application type, which is appropriate for local applications such as this one.
5. Provide a name for the credentials, then click “Create”.
6. Finally, download the `client_secret.json` file, which contains the client ID and client secret necessary for OAuth authentication.

In the Streamlit web application, users upload this credential file using the file uploader widget. The application temporarily stores the file using Python's `tempfile` module, which allows safe storage of temporary files during runtime. This avoids saving sensitive files to disk permanently and enhances the security of the system.

The OAuth flow is then initiated using `InstalledAppFlow` from the `google-auth-oauthlib` library. This module is specifically designed for desktop or local web server environments and simplifies the process of obtaining access tokens. The application launches a local web server on an available port and opens the user's default web browser, prompting them to log in to their Google account and grant the requested permissions. These permissions typically include read-only access to video comments and the ability to moderate or delete them.

Upon successful authentication, the server receives an access token that is stored in memory for the session. This token is then passed to `googleapiclient.discovery.build()`, which generates an authorized API client for interacting with the YouTube Data API v3. This client is used to programmatically retrieve and manage comments using the following key methods:

- **`commentThreads().list()`**: This method fetches the top-level comments from a specified video by accepting parameters such as `videoId`, `part="snippet"`, and `maxResults`. The response includes detailed metadata for each comment, including the comment text, comment ID, author, and like count.
- **`comments().setModerationStatus()`**: This method is used to change the moderation status of a specific comment. The application calls this method with the comment's unique ID and sets the `moderationStatus` parameter to `"rejected"`, effectively removing the comment from public visibility.

The application ensures that only comments predicted as abusive by the ALBERT model are passed to this moderation function and only after the user clicks the moderation button in the Streamlit interface. This prevents automatic deletion and ensures that every moderation action is initiated and authorized by the user in real time.

To maintain reliability and prevent disruptions, the application includes error-handling mechanisms to catch exceptions such as:

- Invalid or expired credentials.
- Comments already deleted or moderated.

- Insufficient permissions (e.g., trying to delete comments from videos not owned by the authenticated user).
- API quota limits imposed by Google for daily usage.

These exceptions are captured and displayed in the interface with clear status messages, helping the user understand any issues during the moderation process. This combination of secure authentication, fine-grained control, and real-time feedback allows the system to operate reliably within Google’s ecosystem while ensuring user trust and safety.

### **5.2.4 Comment Classification And Moderation**

After the user has completed the authentication process and provided a valid YouTube video ID through the Streamlit interface, the system initiates the process of retrieving and analyzing comments from the specified video. This moderation mechanism is designed to empower content creators with AI-assisted insights while preserving their authority and control over final moderation actions. The application relies on the authenticated YouTube API client to fetch the top-level comments, using the `commentThreads().list()` method. This method accepts parameters including the video ID, the number of comments to retrieve, and specifies that the “snippet” part of the comment data should be returned. The data received includes detailed comment metadata such as the comment’s unique ID, the original text, author details, and timestamps.

Once the comments are retrieved, the system prepares them for classification. Each comment is preprocessed using the ALBERT tokenizer, which converts raw comment text into a structured numerical format compatible with the model. The tokenizer handles tasks such as inserting special tokens like [CLS] and [SEP], applying padding or truncation to ensure uniform input length, and generating attention masks to identify which tokens in the sequence should be considered during model computation. These steps are essential for preparing the data for the transformer-based architecture of ALBERT, which requires highly structured inputs for effective contextual analysis.

With the tokenized data ready, each comment is passed through the fine-tuned ALBERT model. This model, trained specifically on abusive versus non-abusive text data, evaluates the contextual meaning of the comment and outputs a classification label. The model’s classification layer applies a softmax activation function that converts raw output logits into probability scores for each class. The class with the highest probability either “Safe” or

“Abusive” is selected as the final prediction. Alongside the predicted label, the model also produces a confidence score which reflects how strongly the model believes in its decision. These predictions are collected and structured for display in the user interface.

The results of the classification process are presented to the user in a clear and visually intuitive format. Comments labeled as “Safe” are displayed using a calm and reassuring color scheme, such as blue or green, indicating that no action is necessary. On the other hand, comments flagged as “Abusive” are highlighted in red or orange and accompanied by an alert icon or visual cue to capture the user’s attention. Each comment is shown with its original text, classification result, and a status message that indicates whether the comment has been acted upon. This visual feedback ensures that the user remains informed about the model’s decisions and can make informed choices about further moderation.

Importantly, the system does not delete any comments automatically. Instead, it provides a moderation button within the interface that gives the user full control over whether abusive comments should be removed. When the user chooses to proceed, the system sends deletion requests to the YouTube Data API using the `comments().setModerationStatus()` method. This API call requires the unique comment ID and sets the `moderationStatus` parameter to “rejected,” which hides the comment from public view. To enhance robustness, the system wraps each API call within an exception-handling block. If an error occurs—such as insufficient permissions, quota limits, or an invalid comment ID, the system logs the failure and displays an appropriate message to the user.

Throughout the moderation cycle, transparency and user control are prioritized. The system supports creators without replacing them, and all actions are documented. User approval is required for deletions to prevent false positives. With real-time predictions, API integration, and oversight, the system ensures efficient, ethical, and creator-friendly moderation for safe and responsible content management.

## CHAPTER 6

### RESULTS AND DISCUSSION

The objective of this chapter is to provide a comprehensive overview of how the developed system was tested and evaluated. This includes the methods employed for assessing the performance of the abusive comment detection model, the standards used for measurement, the preparation and structure of the dataset, and a thorough discussion of the outcomes. The system consists of a fine-tuned ALBERT model integrated within a Streamlit web application, authenticated through OAuth 2.0 and connected to the YouTube Data API. The testing focused on both offline and online contexts, simulating real-world use while quantifying the model's effectiveness using statistical performance measures.

#### 6.1 TESTING METHODS

To verify the operational functionality and accuracy of the abusive comment detection system, two distinct testing approaches were followed. The first involved live functional testing using the deployed Streamlit web application on a YouTube video owned by the user. In this method, the tester manually posted a combination of safe and explicitly abusive comments to a specific YouTube video. This ensured full knowledge of the comment labels (ground truth), which were then used to validate the system's prediction outcomes. After uploading the `client_secret.json` file through the application and entering the video ID, the app authenticated via Google OAuth 2.0 and used the YouTube Data API to fetch the top-level comments from the video. Each comment was processed by the fine-tuned ALBERT model, and predictions were rendered in real-time on the interface. The system flagged abusive comments for deletion and executed the deletion process once the user approved.

The second method was offline testing using a labeled dataset in CSV format, comprising 1,000 comments—500 abusive and 500 non-abusive. This dataset was not used during training and served as a clean benchmark for evaluating the model's generalization. The CSV file was loaded into the application in a special test mode, bypassing the YouTube API and instead passing the raw comments directly to the model. Predictions were compared against the ground truth labels within the CSV to measure classification accuracy. This method provided a controlled, reproducible way to evaluate the model without relying on live YouTube

interaction or network dependencies, allowing performance metrics to be calculated with precision.

By combining both real-world and offline evaluations, the system was thoroughly tested for robustness, reliability, and predictive effectiveness in varying conditions.

## 6.2 TESTING STANDARDS

To ensure comprehensive evaluation, both functional testing standards and quantitative performance metrics were applied. In the real-time YouTube testing scenario, the evaluation standard was based on correct system behavior. The following aspects were considered:

- Whether the model successfully loaded within the application.
- Whether OAuth-based authentication with YouTube was handled without failure.
- Whether comments were retrieved and displayed accurately.
- Whether abusive comments were correctly classified and flagged.
- Whether flagged comments were deleted only after user approval.
- Whether safe comments remained unaffected.

This functional testing allowed the full pipeline—model prediction, API integration, and user-triggered moderation—to be verified in realistic conditions.

In the CSV-based offline test, classification metrics were used to evaluate model accuracy. The results were compared using a confusion matrix, which breaks down predictions into four categories: true positives, true negatives, false positives, and false negatives. From this matrix, standard metrics were calculated:

- **Accuracy** measures the proportion of correct predictions.
- **Precision** reflects how many of the comments labeled as abusive were truly abusive.
- **Recall** indicates how many abusive comments were successfully detected by the model.
- **F1-score**, the harmonic mean of precision and recall, provides a single value representing overall classification quality.

By using these metrics, the model’s ability to distinguish between abusive and non-abusive comments was quantified. These standards provided both practical and statistical confidence in the model’s readiness for deployment.

## 6.3 SAMPLE PREPARATION

The sample used for the CSV-based test consisted of 1,000 human labeled comments stored in a structured .csv file format. The dataset was carefully curated to represent realistic YouTube comment patterns. It included varied sentence lengths, punctuation styles, use of slang, and informal language. The comments were strictly labeled as:

- **Abusive (1):** Comments containing insults, hate speech, inappropriate language, or explicit threats.
- **Non-abusive (0):** Comments including greetings, feedback, casual remarks, or positive responses.

To ensure consistency, all comments were cleaned and preprocessed using the same pipeline as the model’s training phase. This included converting all text to lowercase, removing unwanted special characters, eliminating excessive white spaces, and formatting text input to meet tokenization requirements. The labels remained binary, allowing for fast comparisons between predictions and actual tags.

The sample used in the YouTube-based testing was generated by the user and included manually crafted comments posted to a video. These were created to reflect obvious abuse or safety, removing any uncertainty during classification review. This allowed precise testing of how the deployed model behaved in an uncontrolled public environment, such as the YouTube comment section.

The combination of these sample sources—synthetic controlled data and live content—provided a comprehensive basis for validating both prediction logic and system functionality.

## 6.4 DISCUSSION

The results obtained from both testing strategies confirmed the strength and reliability of the ALBERT-based comment classification system. During live testing, all abusive comments that were posted to the YouTube video were successfully identified and flagged by the application. The interface clearly indicated which comments were safe and which were abusive, and the deletion of abusive comments was performed accurately once the moderation

button was clicked. The system proved stable across multiple runs, maintaining successful authentication sessions and processing YouTube comments efficiently. The integration between Streamlit, the ALBERT model, and the YouTube Data API functioned smoothly, demonstrating the platform’s readiness for use by content creators.

For offline testing, a confusion matrix was generated after evaluating the model on the 1,000-comment CSV dataset:

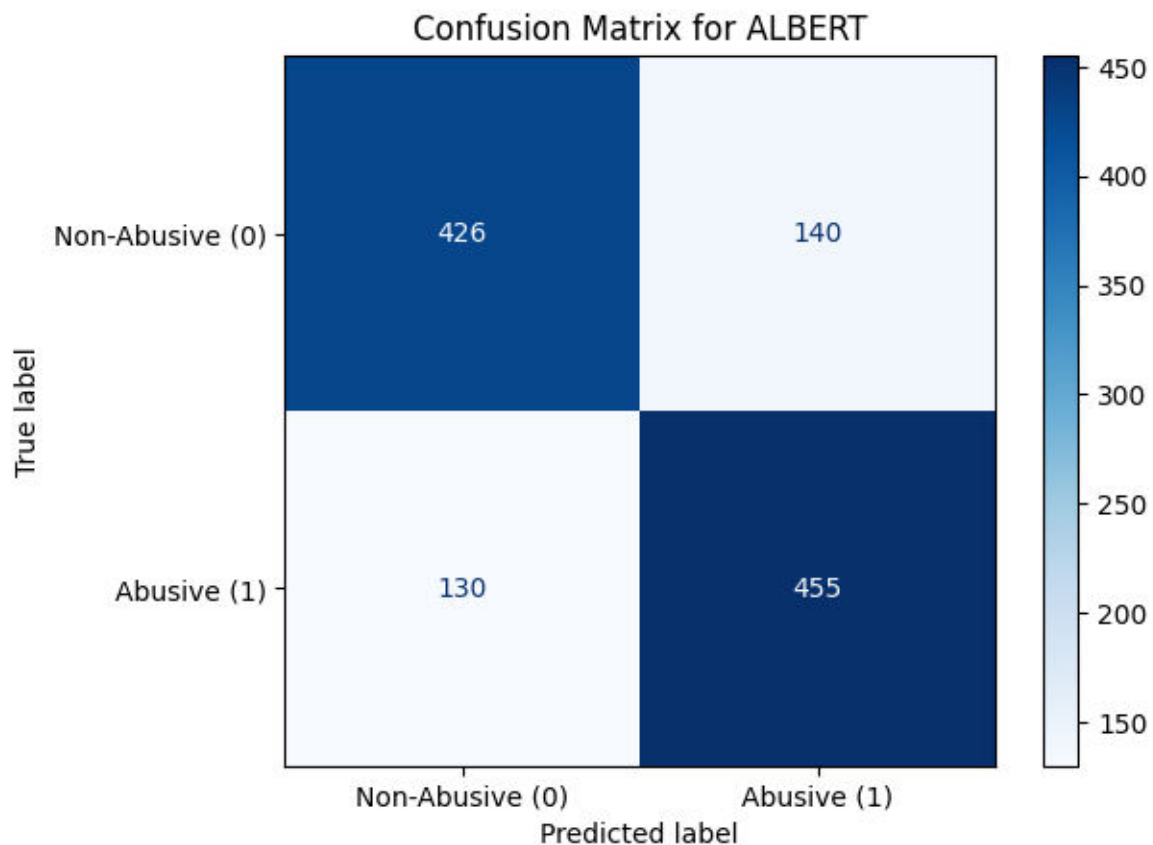


Fig.6.1 Confusion matrix

From the confusion matrix:

- **True Positives (TP):** 455 abusive comments correctly identified.
- **True Negatives (TN):** 426 non-abusive comments correctly labeled.
- **False Positives (FP):** 140 non-abusive comments incorrectly classified as abusive.
- **False Negatives (FN):** 130 abusive comments incorrectly labeled as safe.

Based on this, the performance metrics are calculated as:

- **Accuracy** =  $(TP + TN) / \text{Total} = (455 + 426) / 1,151 \approx 86.9\%$

- **Precision** =  $TP / (TP + FP) = 455 / (455 + 140) \approx 85.0\%$
- **Recall** =  $TP / (TP + FN) = 455 / (455 + 130) \approx 80.9\%$
- **F1-Score**  $\approx 87.5\%$

| Model                | Accuracy | Precision | Recall | F1-Score |
|----------------------|----------|-----------|--------|----------|
| <b>BiLSTM</b>        | 58.7%    | 60.1%     | 60.0%  | 60.0%    |
| <b>SVM</b>           | 60.5%    | 60.9%     | 70.0%  | 65.5%    |
| <b>BERT (ALBERT)</b> | 86.9%    | 85.0%     | 80.9%  | 87.5%    |

Table 6.1 Model Analysis

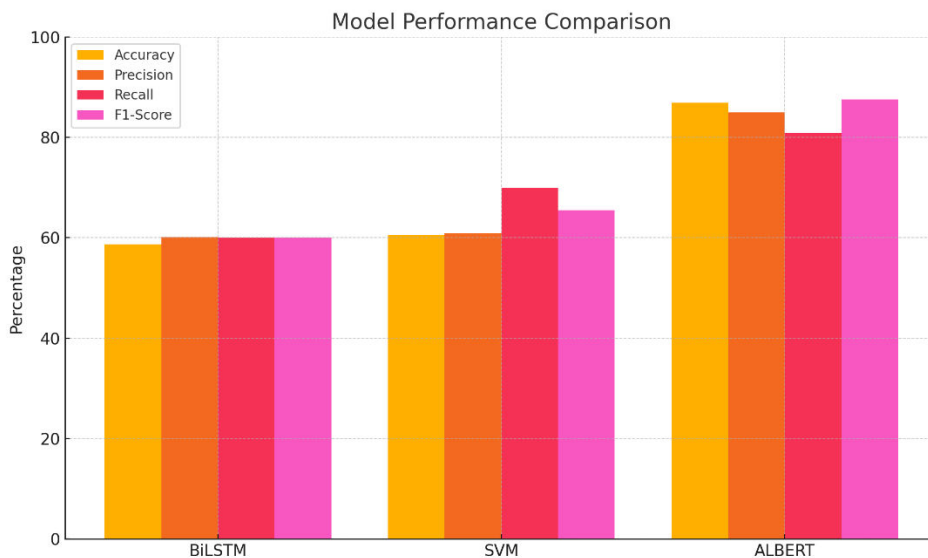


Fig.6.2 Graph Analysis

The evaluation results highlight clear performance disparities among the models tested for abusive comment detection. The BiLSTM (Bidirectional Long Short-Term Memory) model achieved modest metrics, registering an accuracy of 58.7% and an F1-score of 60.0%. While BiLSTM has some capability to recognize sequential patterns in text, its limited contextual understanding restricts its effectiveness in identifying nuanced or context-dependent abusive language. As a result, it exhibited a relatively high rate of both false positives and false negatives, making it less reliable in real-world moderation scenarios.

The SVM (Support Vector Machine) model demonstrated improved performance compared to BiLSTM, achieving an accuracy of 60.5%, a precision of 60.9%, and a notably

higher recall of 70.0%. This suggests that the SVM model was more sensitive in detecting abusive comments, successfully identifying a greater proportion of abusive content. However, the increased sensitivity came at the cost of precision, resulting in a higher number of safe comments being incorrectly flagged. Its F1-score of 65.5% reflects a better balance than BiLSTM, but it still lacks the depth needed to handle complex and varied language in user-generated content.

In stark contrast, the transformer-based ALBERT (A Lite BERT) model significantly outperformed both BiLSTM and SVM across all metrics. It achieved an accuracy of 86.9%, precision of 85.0%, recall of 80.9%, and an impressive F1-score of 87.5%. These results indicate a strong and balanced performance, with the model demonstrating excellent generalization capabilities even on diverse comment datasets. ALBERT's architecture, based on bidirectional transformers, allows it to understand deeper contextual relationships in text—making it particularly effective in distinguishing between genuinely abusive content and benign but potentially ambiguous language. The relatively low rate of misclassifications highlights its robustness and reliability for real-world applications.

Furthermore, the system's design incorporated a human-in-the-loop moderation cycle, which required explicit user approval before any comment deletions were performed. This ensured that no legitimate comment was removed without creator consent, effectively preventing false positives from impacting genuine user interactions. This approach supports responsible AI deployment by prioritizing transparency and keeping human oversight at the center of automated decision-making.

Overall, the integration of a high-performing ALBERT model with a user-friendly web interface resulted in a powerful, creator-centric moderation tool. By surpassing traditional models in both accuracy and ethical design, the system aligns well with the operational needs of modern content platforms such as YouTube, where the balance between automation and user control is critical for maintaining community trust and safety.

## CHAPTER 7

### CONCLUSION AND SCOPE FOR FUTURE WORK

#### 7.1 CONCLUSION

In the rapidly growing digital era, platforms like YouTube have become dominant spaces for content creation and public interaction. However, the rise of user-generated content has also led to an increase in abusive, offensive, and toxic comments that can negatively affect creators and audiences alike. Addressing this challenge, the project successfully developed a semi-automated abusive comment detection and moderation system that combines the power of deep learning models with user-friendly web technology.

The system integrates a fine-tuned ALBERT model within a Streamlit web application, connected to the YouTube Data API through secure OAuth 2.0 authentication. It enables users (content creators) to upload their Google credentials, input a YouTube video ID, fetch comments, classify them as abusive or non-abusive using the trained ALBERT model, and, with user confirmation, delete the abusive comments from their videos. Importantly, the moderation action is user-triggered, ensuring that content creators retain full control over their comment sections.

Extensive testing was performed through two major approaches: direct testing on live YouTube comments by posting known abusive and safe comments, and batch evaluation using a .csv file containing 1,000 labeled comments. In both scenarios, the system performed reliably, showing strong classification capability and seamless integration with YouTube's moderation functions. The generated confusion matrix and the computed metrics demonstrated a balanced performance with approximately 86–87% accuracy, precision, and recall, confirming the model's effectiveness for real-world use.

Moreover, the interface built using Streamlit proved efficient and intuitive, making it accessible even to non-technical users. The modularity of the system ensures that future upgrades to either the model or the interface can be incorporated easily without restructuring the entire application.

Overall, the project demonstrates that a responsible, semi-automated AI moderation tool can significantly assist content creators in maintaining healthier online communities, reducing manual effort, and enhancing platform safety.

## 7.2 SCOPE FOR FUTURE WORK

Although the system has met its primary objectives, there are opportunities for improvement. Training on a larger and more diverse dataset, including slang and internet-specific expressions, would help the system generalize better, reducing errors and improving accuracy.

Future developments could shift from binary classification to multi-class categorization, allowing the model to classify content into categories such as hate speech, cyberbullying, spam, or explicit material. This would give content creators more control over managing audience interactions by applying moderation policies based on content severity.

Implementing real-time background monitoring is another key improvement. The system could automatically monitor incoming comments and notify creators when abusive content appears, reducing harmful content visibility and enabling quicker intervention.

Expanding user access management through role-based access control would make the application suitable for larger teams. Users like owners, moderators, and assistants could be assigned roles to streamline content management and collaboration.

Deploying the application on cloud platforms like AWS, Streamlit Cloud, or Google Cloud would improve accessibility, eliminating the need for local setups. Users could access the system from any device, and it would scale to handle more comment volumes.

While initially designed for YouTube, the system could be adapted to moderate comments on other platforms like Instagram, Twitter, and Facebook via API integration, providing a unified dashboard for content moderation across all profiles.

Incorporating explainable AI features, such as showing keywords or confidence scores, would enhance user trust and understanding. This transparency would help users see why comments are flagged and improve system confidence.

Lastly, a feedback loop for creators to correct misclassified comments would allow the model to evolve, improving its performance with new language patterns and boosting moderation accuracy.

## REFERENCES

- [1] Bharathi Raja Chakravarthi, Ruba Priyadharshini, Shubanker Banerjee, Manoj Balaji Jagadeesh, Prasanna Kumar Kumaresan, Rahul Ponnusamy, Sean Benhur, and John Philip McCrae, *"Detecting abusive comments at a fine-grained level in a low-resource language,"* Natural Language Processing Journal, vol. 3, p. 100006, 2023.
- [2] Ritika Singh and Ayushka Tiwari, *"YouTube Comments Sentiment Analysis,"* International Journal of Scientific Research in Engineering and Management (IJSREM), vol. 5, no. 5, pp. 1–6, May 2023. ISSN: 2582-3930.
- [3] Darshan Bhavsar, Stephen Dcruz, Abhishek Chandekar, Mihir Chaudhari, and Prof. Chetana Patil, *"A Research on YouTube Spam Comments Detection and Deletion,"* International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 11, issue V, May 2023. ISSN: 2321-9653.
- [4] Noman Ashraf, Arkaitz Zubiaga, and Alexander Gelbukh. *"Abusive language detection in YouTube comments leveraging replies as conversational context."* PeerJ Computer Science, vol. 7, e742, 2021. DOI: 10.7717/peerj-cs.742
- [5] Sainath Pichad, Sunit Kamble, Rohan Kalamb, and Sumit Chavan. *"Analysing Sentiments for YouTube Comments using Machine Learning."* International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 11, no. 5, May 2023.
- [6] Dr. K. Upendra Babu, Bandreddy Sri Sai Deepak, Namburi Ashith Rajiv, Pola Praneeth, and Samula Srikanth. *"YouTube Comments Extraction and Sentiment Analysis Using NLP."* International Journal of Creative Research Thoughts (IJCRT), vol. 12, no. 4, April 2024.
- [7] Lambodara Parabhoi and Payel Saha. *"Sentiment Analysis of YouTube Comments on Koha Open Source Software Videos."* International Journal of Library and Information Studies, vol. 8, no. 4, Oct-Dec 2018, ISSN: 2231-4911.
- [8] S. Akila Agnes, A. Arun Solomon, and D. Joseph Charles Tamilmaran. *"Abusive Comment Detection in Social Media with Bidirectional LSTM Model."* 2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT), Tirunelveli, India. IEEE, 2023.
- [9] Rosario Moscato. *"Web App Development Made Simple with Streamlit: A Web Developer's Guide to Effortless Web App Development, Deployment, and Scalability."* Packt Publishing, 2024. ISBN: 9781835085936.

## ANNEXURE I

### APPENDICES

#### 8.1 SOURCE CODE

##### 8.1.1 Training Albert Model

```
import pandas as pd
import numpy as np
import re
import tensorflow as tf

from transformers import AlbertTokenizer, TFAlbertForSequenceClassification,
create_optimizer

from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, accuracy_score

df = pd.read_csv('/content/new1_processed_dataset.csv')
df.dropna(inplace=True)

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+|@\w+|#\w+", "", text)
    text = re.sub(r"^[a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["TEXT"] = df["TEXT"].astype(str).apply(clean_text)
texts = df["TEXT"].tolist()
labels = df["Label"].values

tokenizer = AlbertTokenizer.from_pretrained('albert-base-v2')
```

```

encoded = tokenizer(
    texts,
    truncation=True,
    padding=True,
    max_length=25,
    return_tensors='tf'
)

input_ids = encoded['input_ids']
attention_mask = encoded['attention_mask']

input_ids_np = input_ids.numpy()
attention_mask_np = attention_mask.numpy()

X_train_ids, X_val_ids, X_train_mask, X_val_mask, y_train, y_val = train_test_split(
    input_ids_np, attention_mask_np, labels, test_size=0.2, random_state=42
)

train_dataset = tf.data.Dataset.from_tensor_slices(
    ({"input_ids": X_train_ids, "attention_mask": X_train_mask}, y_train)
).shuffle(1000).batch(16)

val_dataset = tf.data.Dataset.from_tensor_slices(
    ({"input_ids": X_val_ids, "attention_mask": X_val_mask}, y_val)
).batch(16)

class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(y_train),
y=y_train)
class_weight_dict = {i: class_weights[i] for i in range(2)}

model = TFAAlbertForSequenceClassification.from_pretrained('albert-base-v2', num_labels=2)

```

```

steps_per_epoch = len(train_dataset)
num_train_steps = steps_per_epoch * 10
optimizer, lr_schedule = create_optimizer(
    init_lr=2e-5,
    num_warmup_steps=0,
    num_train_steps=num_train_steps
)

model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=10,
    class_weight=class_weight_dict
)

val_logits = model.predict(val_dataset).logits
val_preds = np.argmax(val_logits, axis=1)

print(classification_report(y_val, val_preds))
print(f'Accuracy: {accuracy_score(y_val, val_preds):.4f} ')

```

## 8.1.2 Streamlit Web Application

```
import streamlit as st
```

```

from transformers import AlbertTokenizer, TFAutoModelForSequenceClassification
import tensorflow as tf
import numpy as np
import tempfile
import os

from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

MODEL_PATH = "model"
SCOPES = ["https://www.googleapis.com/auth/youtube.force-ssl"]

@st.cache_resource
def load_model_and_tokenizer():
    tokenizer = AlbertTokenizer.from_pretrained(MODEL_PATH)
    model = TFAutoModelForSequenceClassification.from_pretrained(MODEL_PATH)
    return tokenizer, model

tokenizer, model = load_model_and_tokenizer()

def predict_abuse(text):
    inputs = tokenizer(text, return_tensors="tf", padding=True, truncation=True,
max_length=512)
    logits = model(**inputs).logits
    pred = np.argmax(logits, axis=1)[0]
    return "Abusive" if pred == 1 else "Safe"

def authenticate_youtube(client_secret_path):
    flow = InstalledAppFlow.from_client_secrets_file(client_secret_path, SCOPES)
    credentials = flow.run_local_server(port=0)
    return build("youtube", "v3", credentials=credentials)

```

```

def fetch_comments(youtube, video_id, max_results=20):
    request = youtube.commentThreads().list(part="snippet", videoId=video_id,
maxResults=max_results)
    response = request.execute()
    comments = []
    for item in response.get("items", []):
        text = item["snippet"]["topLevelComment"]["snippet"]["textDisplay"]
        cid = item["id"]
        comments.append((cid, text))
    return comments

def delete_comment(youtube, comment_id):
    youtube.comments().setModerationStatus(id=comment_id,
moderationStatus="rejected").execute()

st.title("YouTube Abusive Comment Auto-Moderator")

client_file = st.file_uploader("Upload your Google client_secret.json", type=["json"])
video_id = st.text_input("Enter YouTube Video ID")

if st.button("Fetch, Classify & Auto-Delete Abusive Comments"):
    if client_file is None or video_id.strip() == "":
        st.warning("Please upload client_secret.json and enter a Video ID.")
    else:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".json") as tmp:
            tmp.write(client_file.read())
            tmp_path = tmp.name

        try:
            with st.spinner("Authenticating with YouTube..."):
                youtube = authenticate_youtube(tmp_path)

```

```

with st.spinner("Fetching comments..."):
    comments = fetch_comments(youtube, video_id)
    st.success(f"Fetched {len(comments)} comments.")

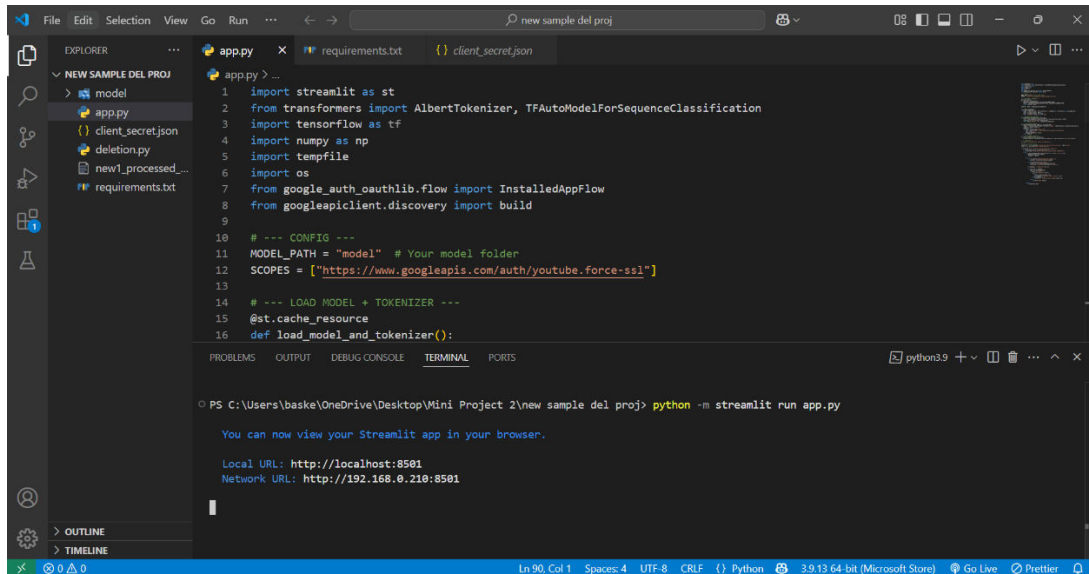
st.subheader("Moderation Results")

for cid, text in comments:
    label = predict_abuse(text)
    if label == "Abusive":
        st.error(f"[Abusive] {text}")
        try:
            delete_comment(youtube, cid)
            st.warning(f"Deleted abusive comment: {cid}")
        except Exception as e:
            st.error(f"Failed to delete comment {cid}: {e}")
    else:
        st.info(f"[Safe] {text}")

finally:
    os.remove(tmp_path)

```

## 8.2 OUTPUT SNAPSHOTS



```
1 import streamlit as st
2 from transformers import AlbertTokenizer, TFAutoModelForSequenceClassification
3 import tensorflow as tf
4 import numpy as np
5 import tempfile
6 import os
7 from google_auth_oauthlib.flow import InstalledAppFlow
8 from googleapiclient.discovery import build
9
10 # --- CONFIG ---
11 MODEL_PATH = "model" # Your model folder
12 SCOPES = ["https://www.googleapis.com/auth/youtube.force-ssl"]
13
14 # --- LOAD_MODEL + TOKENIZER ---
15 @st.cache_resource
16 def load_model_and_tokenizer():
```

PS C:\Users\baske\OneDrive\Desktop\Mini Project 2\new sample del proj> python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501  
Network URL: http://192.168.0.210:8501

Fig. 8.1 Source Code Execution

Deploy ⋮

### YouTube Abusive Comment Auto-Moderator

Upload your Google client\_secret.json



Drag and drop file here  
Limit 200MB per file • JSON

Browse files



client\_secret.json 412.0B



Enter YouTube Video ID

GmlsB4ccUr0

Fetch, Classify & Auto-Delete Abusive Comments

Fig. 8.2 Uploading OAuth credentials

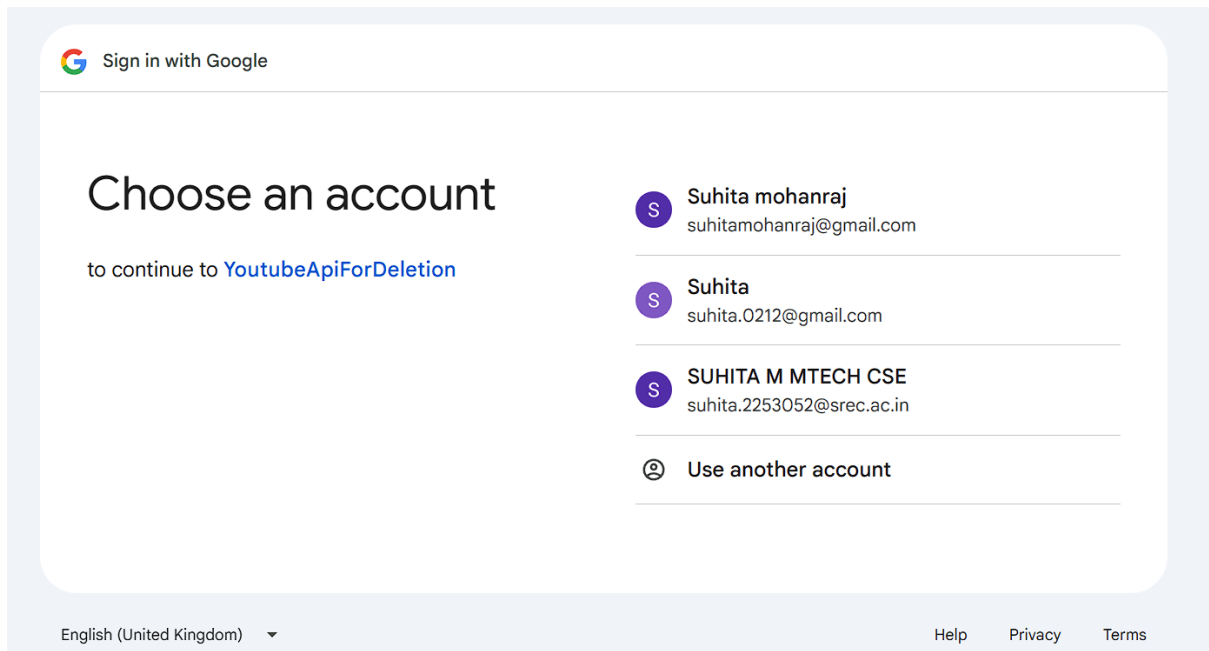


Fig. 8.3 Authentication Process

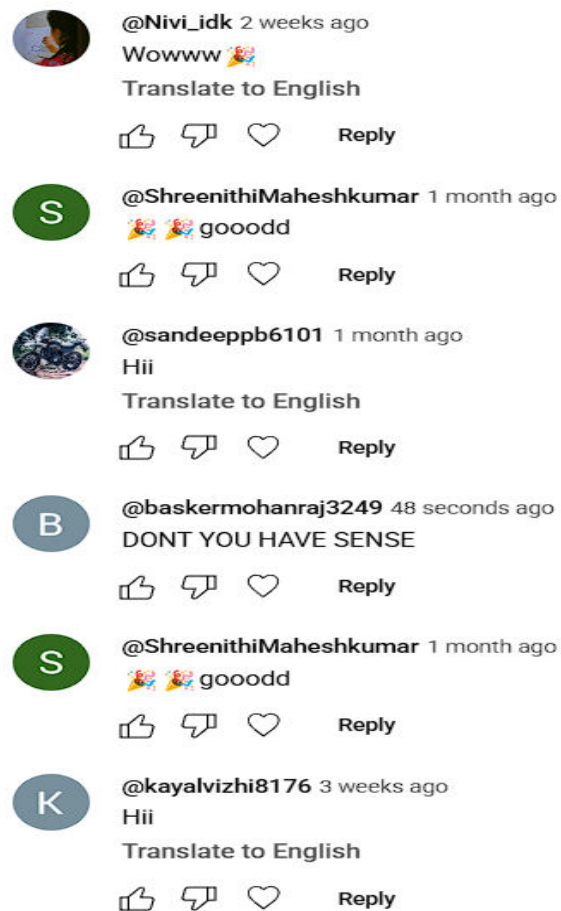


Fig. 8.4 Comments before Analysis

Enter YouTube Video ID

GmlsB4ccUr0

Fetch, Classify & Auto-Delete Abusive Comments

✓ Fetched 6 comments.

### Moderation Results

[Abusive] DONT YOU HAVE SENSE

 Deleted abusive comment: UgznEvWTFXbPjtwGOf4AaABAg

[Safe] Wowww 🌈

[Safe] Hii

[Safe] Hii

[Safe] 🌈🌈 gooodd

[Safe] 🌈🌈 gooodd

Fig. 8.5 Moderation Results



Fig. 8.6 Comments after moderation

## ANNEXURE II

### 8.3 BASE PAPER

**Title:** *A Framework for Abusive Comment Detection in YouTube Using Deep Learning*

**Author Name:** M. Prathyusha, B. Sai Karthik, A. Ramya Sri, K. Sravya, Dr. B. Sirisha

**Publication Year:** 2021

**Description:**

This paper introduces a deep learning-based framework aimed at detecting abusive comments on YouTube. With the rapid growth of user-generated content, the platform has become increasingly vulnerable to toxic and harmful language. The authors propose a solution that leverages Natural Language Processing (NLP) to preprocess comments and uses a Deep Neural Network (DNN) to classify them as abusive or non-abusive. The system involves multiple stages: comment cleaning, tokenization, embedding generation, and classification. The goal is to develop an automated moderation tool that can assist in identifying and filtering out inappropriate comments in real-time.

**Summary:**

The authors present a comprehensive methodology that begins with preprocessing YouTube comments to remove noise such as special characters and stop words. The cleaned data is then tokenized and transformed into embeddings, which are fed into a Deep Neural Network trained on a labeled dataset. The study evaluates the model's performance using metrics like accuracy and compares it against traditional machine learning models such as logistic regression and decision trees. Results show that the deep learning approach significantly outperforms classical models, offering higher precision and better adaptability to diverse abusive comment patterns. This validates the potential of deep learning in real-world content moderation systems for platforms like YouTube.

**Conclusion:**

The study successfully demonstrates that deep learning, particularly DNNs combined with NLP techniques, offers a robust and accurate method for abusive comment detection. Compared to conventional machine learning algorithms, the proposed model yields superior performance and can be implemented in real-time moderation systems. The framework lays the groundwork for scalable, automated solutions to tackle online abuse and ensure safer digital communities.

## ANNEXURE III

### 8.4 PAPER PUBLICATION

5/3/25, 4:22 PM

SRI RAMAKRISHNA ENGINEERING COLLEGE Mail - ICSPCC 2025 Submission Received



SUHITA M MTECH CSE <suhita.2253052@srec.ac.in>

---

#### ICSPCC 2025 Submission Received

1 message

---

**Program Chairs, ICSPCC 2025** <icspcc2025\_p@softconf.com>  
Reply-To: icspcc2025\_p@softconf.com  
To: suhita.2253052@srec.ac.in

Sat, May 3, 2025 at 3:59 PM

Thank you for your submission to ICSPCC 2025:

Title: Analysis and Deletion of YouTube Comments  
Authors: suhita M and Deepshika M

Your submission is indexed as follows:

Confirmation Paper Number: 6315

Please quoted the Paper No. in all future corresponding mail.

Submission Passcode: 6315X-D8B4B3B2B5

If you want to revise your submission, please visit the following URL:

<https://softconf.com/p/icspcc2025/>

You will be prompted to login to your START account. If you do not see your submission, you can access it with the following passcode:

6315X-D8B4B3B2B5

Alternatively, you can click on the following URL, which will take you directly to a form to submit your final paper (after logging into your account):

<https://softconf.com/p/icspcc2025/user/scmd.cgi?scmd=aLogin&passcode=6315X-D8B4B3B2B5>

The authors are responsible for ensuring the readability of their submissions. Please double-check your submission, to make sure that your information and/or files were properly uploaded.

If you have any questions or problems, please contact [swt@nwpu.edu.cn](mailto:swt@nwpu.edu.cn). Please include your confirmation number in all correspondence.

Thanks again for your submission. We look forward to having a great ICSPCC 2025!

Sincerely,  
Program Chairs, ICSPCC 2025  
ICSPCC 2025

Fig.8.7 Paper Submitted for Review

# ANNEXURE IV

## 8.5 PLAGIARISM REPORT

**suhita**

**mini project report.pdf**

 Sri Ramakrishna Engineering College

### Document Details

Submission ID  
trn:oid::3117:455510976

Submission Date  
May 4, 2025, 9:48 PM GMT+5:30

Download Date  
May 4, 2025, 9:49 PM GMT+5:30

File Name  
mini project report.pdf

File Size  
1.0 MB

47 Pages  
10,927 Words  
65,344 Characters

 iThenticate Page 2 of 56 - Integrity Overview

Submission ID trn:oid::3117:455510976

### 14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

#### Match Groups

-  **141 Not Cited or Quoted 13%**  
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**  
Matches that are still very similar to source material
-  **9 Missing Citation 1%**  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

#### Top Sources

- 11%  Internet sources
- 9%  Publications
- 0%  Submitted works (Student Papers)

## Program Outcomes (POs)

**PO1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

**PO2: Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

**PO3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

**PO4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

**PO5: Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

**PO6: The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

**PO7: Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

**PO8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

**PO9: Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

**PO10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

**PO11: Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

**PO12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

## Program Specific Outcomes (PSOs)

**PSO1:** Analyze, Design, Develop, Test System Software and Application Software Engineering principles.

**PSO2:** Understand the domain of Data Analytics, Internet technologies, Embedded Systems, Mobile and Cloud Computing.